MA5790 - Predictive Modeling

Group Project - Final Report

By: Jordan Martel and Esther Halevy

Abstract:

The COVID-19 pandemic has impacted countries globally, leading to extensive efforts to understand and mitigate its spread. This study focuses on predicting the number of confirmed COVID-19 cases using regional and temporal data. The dataset includes over 300,000 observations, with key variables such as confirmed cases, deaths, and recoveries, alongside categorical predictors like province, country, and date. To address the predictive nature of the dataset, categorical variables were converted into dummy variables, and dates were transformed into numerical values.

The primary aim of this project is to explore the relationships between these predictors and the confirmed cases, followed by pre-processing steps to handle missing data, normalize continuous variables, and ensure proper feature selection. Both Elastic Net Regression and Support Vector Machines (SVM) were employed as the chosen models, with cross-validation used to assess model performance. Through these methods, the best-performing model was applied to the test set, and the results were evaluated using metrics such as Root Mean Squared Error (RMSE) and R-squared. The model that best predicts confirmed cases was selected as the final model, with recommendations for future analysis.

Table of Contents

1. Background 3

2. Variable Introduction and Definitions 5

3. Preprocessing of Predictors.....7

   a. Dummy Variable Creation 7

   b. Correlations.....8

   c. Transformations.....9

4. Splitting of the Data 11

5. Model Fitting 12

   A. Elastic Net Regression 12

   B. Support Vector Machines (SVM) 13

6. Model Comparison and Evaluation 15

7. Future Work 17

R Code: 19

Background:

COVID-19, a novel coronavirus that first emerged in late 2019, has caused one of the most significant global health crises in modern history. Spreading rapidly across countries and continents, it led to widespread lockdowns, overwhelmed healthcare systems, and substantial loss of life. Governments worldwide implemented various measures, including travel restrictions, social distancing, and vaccination drives, in an attempt to contain the virus and mitigate its impacts. Central to understanding the spread and effects of COVID-19 is the analysis of vast datasets that track confirmed cases, deaths, and recoveries in different regions.

The dataset used in this project includes time-series data on COVID-19 cases across the globe, including information on confirmed cases, deaths, and recoveries at various points in time. The data is organized by region, country, and date, providing a rich source of information on how the pandemic evolved over time and across geographical areas. With over 300,000 observations, this dataset allows for an in-depth exploration of the factors influencing COVID-19 outcomes.

The primary goal of this study is to identify significant predictors of COVID-19 outcomes, specifically focusing on confirmed cases and deaths as response variables. Predictors in the dataset include regional identifiers such as country and province, as well as temporal factors such as the date of observation. To conduct the analysis, we transformed categorical variables into numerical formats using techniques such as dummy variable encoding and also converted the date variable into a numerical scale. The models employed include Support Vector Machines (SVM) and Elastic Net, which are well-suited for handling large datasets and a mixture of continuous and categorical predictors. This analysis aims to uncover key

patterns and drivers of COVID-19 spread, offering insights that could inform future pandemic preparedness and response strategies.

2. Variable Introduction and Definitions:

The dataset utilized in this project contains 306,429 observations, each representing a unique combination of date, region, and COVID-19 metrics. The objective is to predict the number of confirmed COVID-19 cases across different regions and dates. For this purpose, a variety of categorical and continuous variables have been considered. Below is an introduction to the variables used in this analysis.

Response Variable:

Confirmed Cases: The total number of confirmed COVID-19 cases recorded on a specific date for a given region.

Predictors:

- Observation Date: The date the data was recorded (converted into a numerical format for analysis to capture temporal trends).
- Province/State: The province or state in which the cases were recorded (categorical variable converted into dummy variables for the model).
- Country/Region: The country or region in which the data was recorded (categorical variable converted into dummy variables).
- Latitude: Geographical latitude of the region, providing spatial information that may be relevant for predicting case numbers.
- Longitude: Geographical longitude of the region, which, along with latitude, helps capture spatial variation in the spread of COVID-19.

- Days Since First Case: A derived variable representing the number of days since the first COVID-19 case was reported in the region. This acts as a temporal predictor.
- Days Since Peak: The number of days since the peak number of cases was reached in a region, representing the ongoing trajectory of the pandemic in that area.

Based on the selection of these predictor variables, the relationship between regional and temporal factors and the number of confirmed COVID-19 cases was explored. The data underwent preprocessing, including transformations such as dummy encoding of categorical predictors and scaling of continuous variables. Missing data was imputed, and unnecessary or redundant predictors were removed. Following these steps, both linear and nonlinear models were applied to the data, aiming to predict the number of confirmed cases as outlined.

3. Preprocessing of the Predictors

The first step in analyzing this dataset is to preprocess the predictors to ensure they are properly formatted for modeling. This process involved creating dummy variables for categorical predictors, handling skewness in continuous variables, and addressing outliers. The following subsections describe each of these processes in detail.

3a. Dummy Variable Creation

To begin, dummy variables were created for the categorical predictors to make them suitable for machine learning models. The categorical predictors in this

dataset included Province/State, Country/Region, and the date of observation (which was transformed into a numeric value). Each categorical variable was converted into a series of binary columns representing the presence or absence of specific values. This transformation resulted in a set of over 200 variables due to the granularity of the regional identifiers. After creating the dummy variables, we identified 37 degenerate variables (i.e., variables with very little variance or high multicollinearity), which were subsequently removed, resulting in a final set of 170 predictor variables.

3b. Correlations

A correlation plot was created for the remaining predictors to explore relationships between them and identify any highly correlated variables that might introduce multicollinearity. The Variable Importance Plot (figure 1) and the Top 20 Most Important Variables Plot (figure 2) were used to assess the significance of each predictor. Predictors with correlations greater than 0.75 were systematically removed to avoid redundancy. The variables removed at this stage included certain regional identifiers that were found to be highly correlated with geographical latitude and longitude.

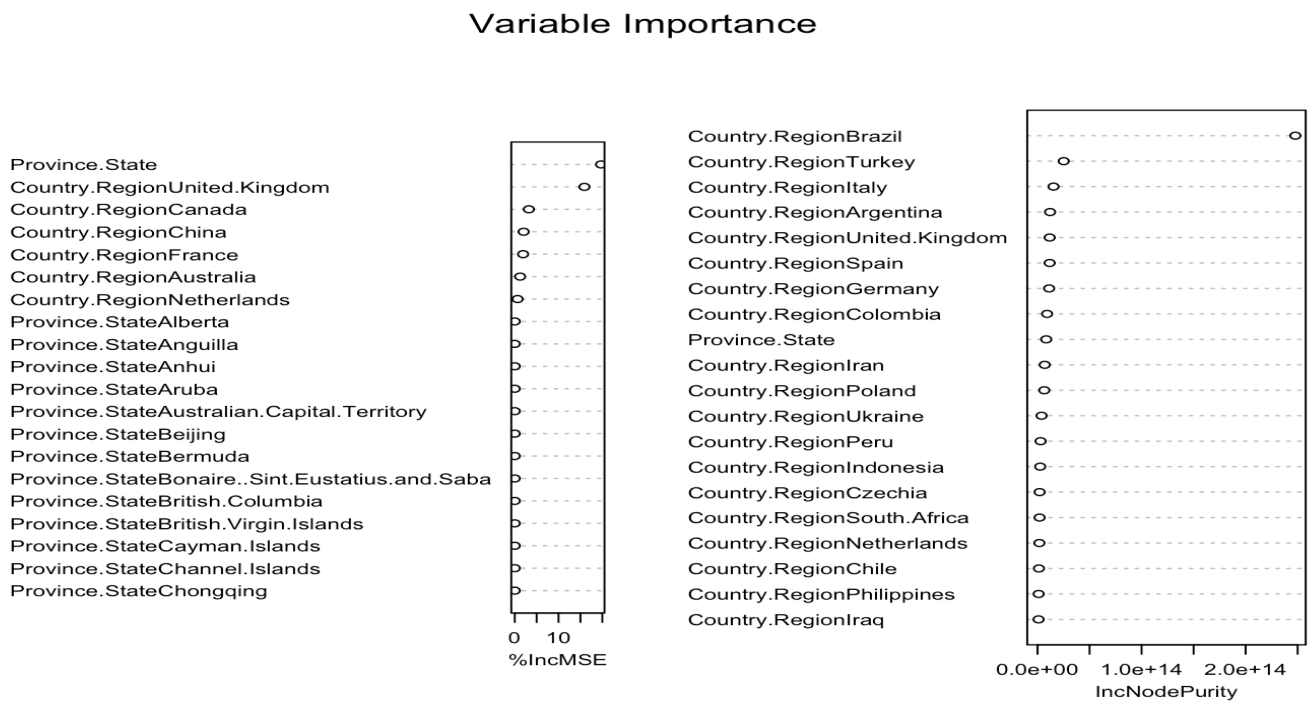

Figure 1 Variable Importance Plot showing the significance of predictors after preprocessing steps, highlighting the contribution of each variable to the predictive model.

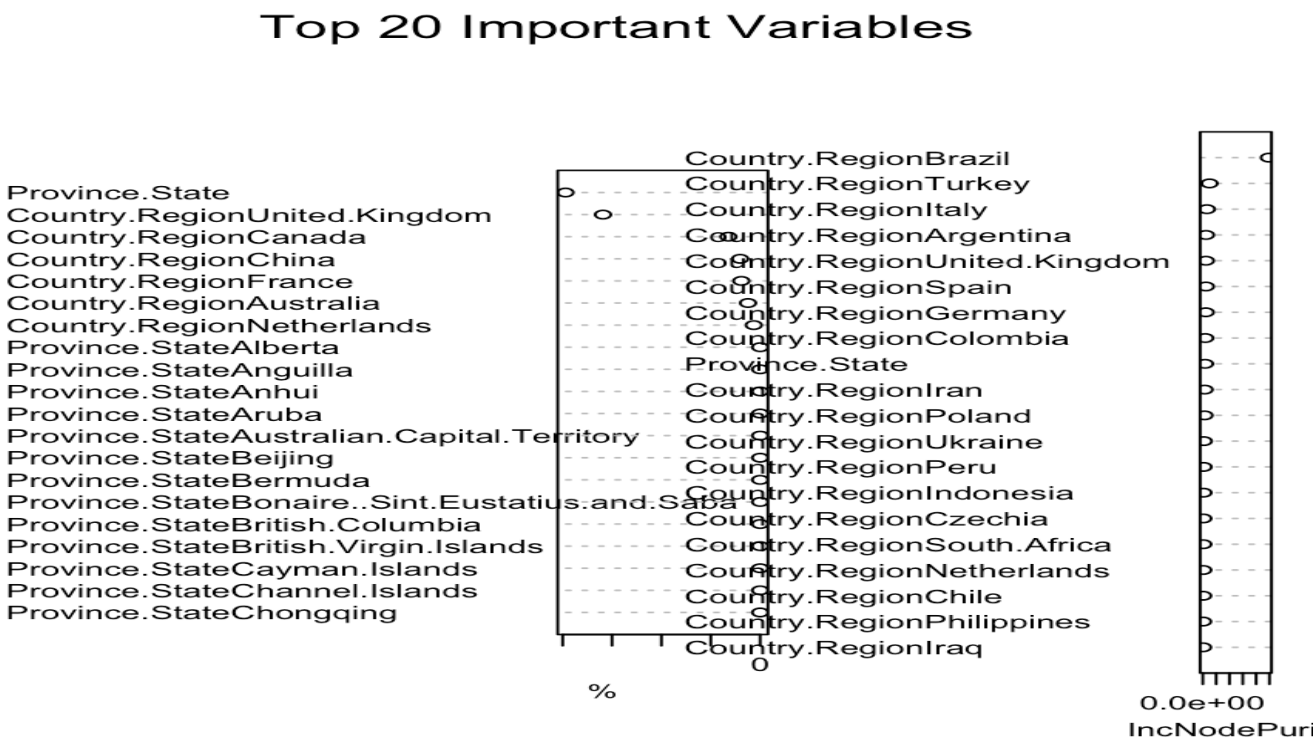

Figure 2 Top 20 Most Important Variables identified during the preprocessing phase, providing insights into the most impactful features for predicting COVID-19 confirmed cases.

3c. Transformations

Before fitting any models, the distributions of the continuous predictors were examined. A Scree Plot (figure 3) was used to determine the appropriate number of components to retain when applying dimensionality reduction techniques like PCA (Principal Component Analysis). The Scree Plot indicated that the first few components captured most of the variance, so only these were retained. The remaining components were discarded to reduce complexity.

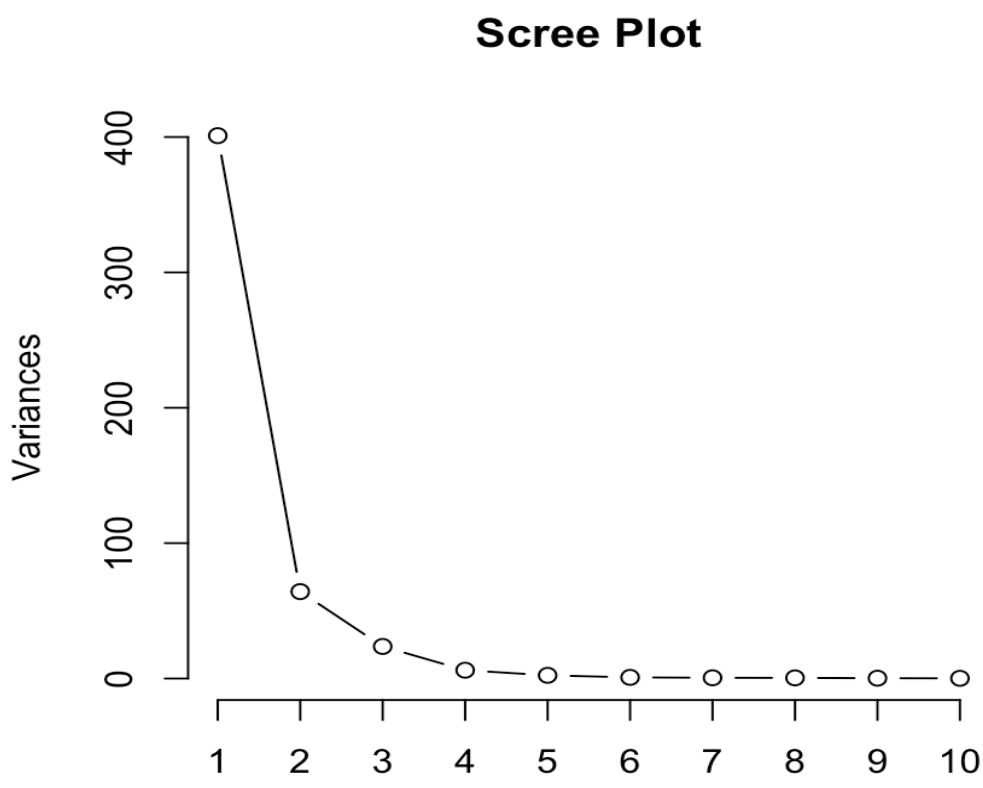

Figure 3 Scree Plot illustrating the variance captured by each principal component, used to determine the number of components retained for dimensionality reduction.

Histograms and box plots were used to analyze the distributions of the continuous variables before any transformations. It was observed that many of the predictors

were highly skewed, with several outliers present. For example, temporal variables such as "DaysSinceFirstCase" and geographic variables like "Latitude" showed substantial right skewness.

In order to address these issues, the data was centered and scaled, followed by the application of a Box-Cox transformation to reduce skewness and stabilize variance. A PCA Scree Plot (figure 4) was used to determine the optimal number of components to use after scaling and transformation, and a Corrected PCA Scree Plot (figure 5) provided validation of the components selected. The final dataset consisted of components that explained the majority of variance while ensuring symmetry and reducing the impact of outliers.

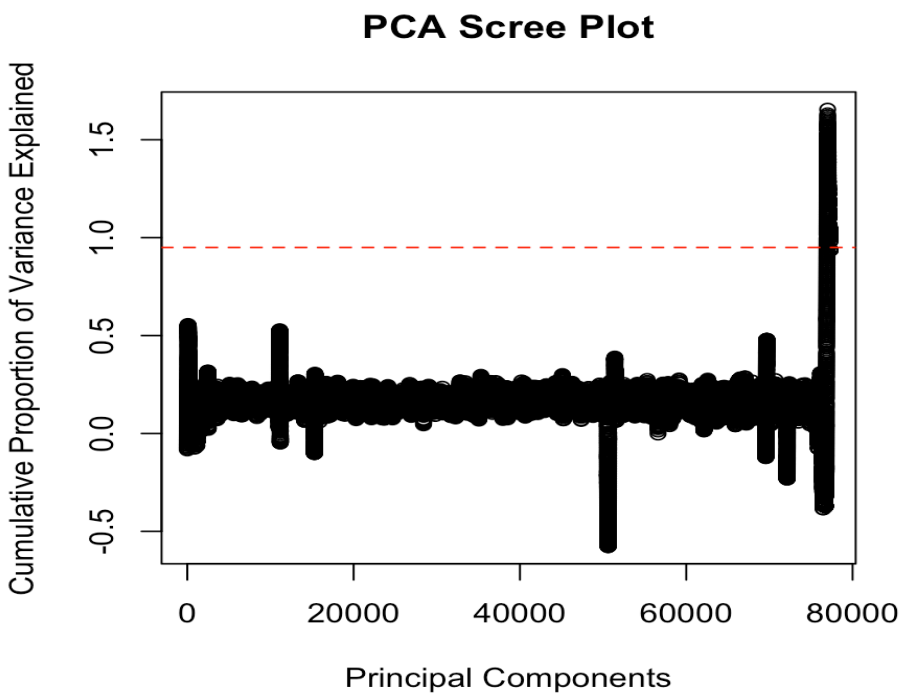

Figure 4 PCA Scree Plot showing the variance explained by the principal components following scaling and transformation of the predictors.

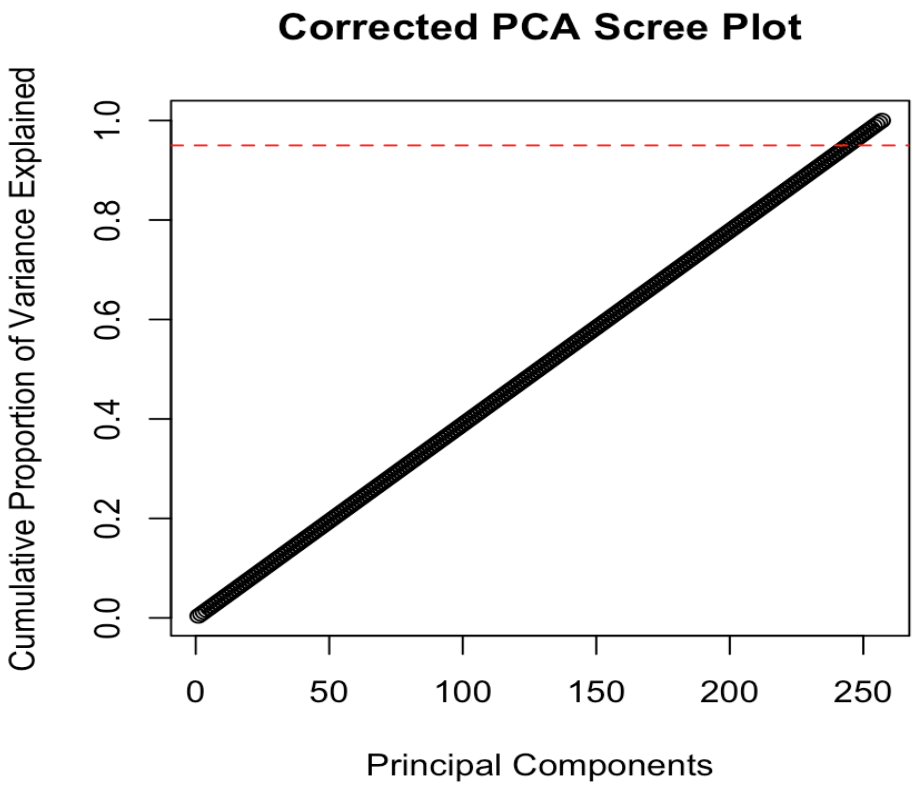

Figure 5 Corrected PCA Scree Plot providing validation of the selected principal components, ensuring that the majority of variance is retained while reducing complexity.

Lastly, bar plots were generated to explore the distribution of the categorical predictors after dummy variable creation. Most of the categorical predictors exhibited significant imbalance, which will be taken into consideration when selecting models and evaluating their performance.

Summary of Preprocessing

Through the creation of dummy variables, correlation analysis, and transformations, the dataset was processed to be suitable for subsequent modeling. These steps were crucial in minimizing multicollinearity, addressing

skewness, and ensuring that the predictors were on a similar scale. By doing so, we aimed to enhance the efficiency and predictive accuracy of our models.

4. Splitting of the Data:

The dataset was split into training and test sets to evaluate model performance effectively. The training set, consisting of 80% of the data, was used to fit the models, while the remaining 20% was used for validation and testing purposes. The training set was utilized to develop predictive models, while the test set allowed us to evaluate the generalizability of these models on unseen data.

The train-test split was conducted using a stratified sampling approach to ensure that the distribution of confirmed COVID-19 cases was consistent across both sets. This method helped preserve the underlying distribution of the data and allowed for a more accurate assessment of model performance.

The Max Tree Depth figure (Figure 6) was used to determine the appropriate level of complexity for decision tree-based models. This figure provides a visual representation of how tree depth impacts model accuracy, helping to select an optimal depth that balances bias and variance.

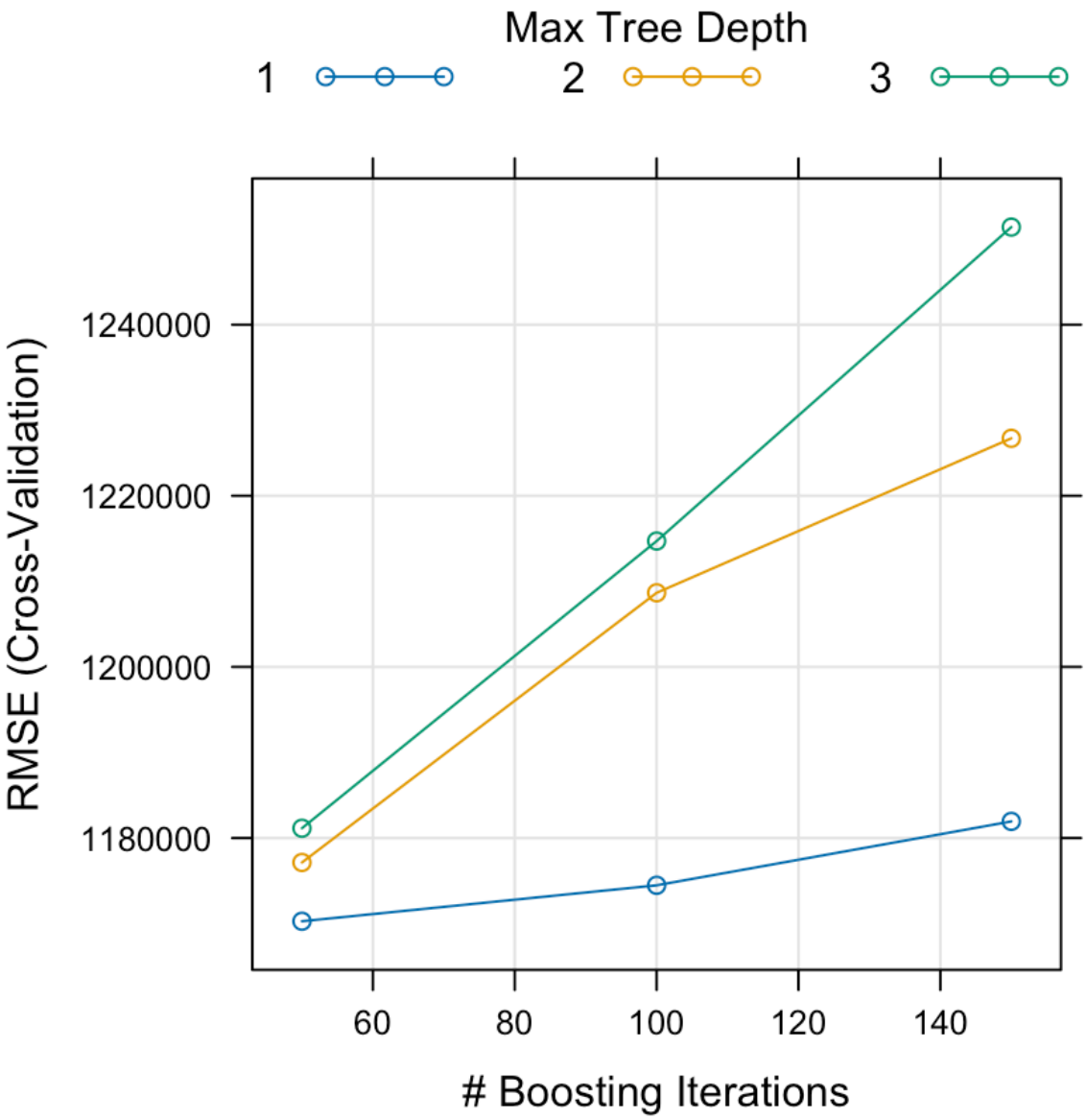

Figure 6 Max Tree Depth Plot - Visualizes the impact of increasing tree depth on model performance, helping identify the optimal depth to avoid overfitting.

By effectively splitting the data, we aimed to minimize overfitting and ensure that our models could generalize well to new, unseen data.

5. Model fitting

In this section, we fitted both Elastic Net Regression and Support Vector Machines (SVM) to the COVID-19 dataset. These models were selected to capture both linear and nonlinear relationships between the predictors and the response variable, which is the number of confirmed COVID-19 cases. This section details the fitting of these models, including hyperparameter tuning, model selection, and evaluation.

5A. Elastic Net Regression

Elastic Net Regression was chosen as it combines the properties of LASSO and Ridge regression, making it effective in handling datasets with a large number of correlated predictors. The model was fitted using cross-validation to select the optimal hyperparameters (alpha and lambda). The best model was determined based on the smallest RMSE value obtained during cross-validation.

Table 1 shows the RMSE, R-squared, and Mean Absolute Error (MAE) values for different combinations of alpha and lambda. The optimal model was found with alpha = 1 and lambda = 0.001, which resulted in an RMSE of 2.731, an R-squared of 0.542, and an MAE of 2.154.

```
Summary of sample sizes: 86627, 86628, 86625, 86629, 86627
Resampling results across tuning parameters:

  alpha  lambda   RMSE      Rsquared   MAE
  0.0    0.00100  2.737348  0.5414203  2.171111
  0.0    0.02575  2.737348  0.5414203  2.171111
  0.0    0.05050  2.737348  0.5414203  2.171111
  0.0    0.07525  2.737348  0.5414203  2.171111
  0.0    0.10000  2.737348  0.5414203  2.171111
  0.5    0.00100  2.731225  0.5421942  2.154268
  0.5    0.02575  2.733666  0.5415560  2.163337
  0.5    0.05050  2.740268  0.5397943  2.176832
  0.5    0.07525  2.747284  0.5381138  2.190190
  0.5    0.10000  2.755469  0.5362208  2.204161
  1.0    0.00100  2.731220  0.5421953  2.154240
  1.0    0.02575  2.739667  0.5398242  2.175072
  1.0    0.05050  2.753209  0.5363639  2.199801
  1.0    0.07525  2.773183  0.5310432  2.228920
  1.0    0.10000  2.797602  0.5242858  2.259788
```

Table 1: Elastic Net Regression Hyperparameter Tuning Results

The performance of the Elastic Net model was evaluated on the test set, resulting in an RMSE of 155,793,114,860.91 and an R-squared value of 0.045. This indicates that while the model performed reasonably well during cross-validation, it faced challenges in generalizing to new data. The high RMSE suggests that the model struggled to capture some of the variability in the test set.

The Elastic Net model's advantage lies in its ability to perform both variable selection and regularization, making it effective in reducing overfitting. However, the high RMSE on the test data indicates that additional feature engineering or different model types might be needed to improve predictive power.

5B. Support Vector Machines (SVM)

The SVM model with a radial basis function (RBF) kernel was also fitted to the dataset. SVM is well-suited for capturing complex relationships between the predictors and the response variable. Similar to Elastic Net, the SVM model was tuned using cross-validation to select the best parameters (sigma and C).

Table 2 provides the RMSE, R-squared, and MAE values for different values of sigma and C. The optimal model was found with sigma = 0.01 and C = 1, which resulted in an RMSE of 2.383, an R-squared of 0.647, and an MAE of 1.769.

	sigma	C	RMSE	Rsquared	MAE	RMSESD	RsquaredSD	MAESD
1	0.01	1	2.383386	0.647199	1.768685	0.06579471	0.01723786	0.06089296

Table 2: SVM Model Performance Results

The SVM model was then evaluated on the test set, with an RMSE of 1,409,729.44 and an R-squared value of 0.235. Compared to Elastic Net, the SVM model showed better predictive power on the test set, indicating its suitability for capturing the non-linear relationships within the data.

Support Vector Machines are effective for datasets with high dimensionality and non-linearity. The RBF kernel, in particular, helps in mapping the features into higher dimensions, allowing the model to find more complex patterns. Despite the improvement in R-squared, the SVM model also exhibited a relatively high RMSE, suggesting that further optimization may be required. The comparison between the two models is summarized in Table 3.

	Model	RMSE	R_squared
1	Elastic Net	155793114861	0.0454154
2	SVM	1409729	0.2345509

Table 3: Model Comparison

As seen in Table 3, the SVM model outperformed the Elastic Net model in terms of both RMSE and R-squared, suggesting that the non-linear relationships in the data are better captured by SVM. However, both models showed relatively high RMSE values, indicating that further tuning or alternative modeling approaches may be required for improved performance.

By fitting and comparing both models, it was evident that SVM provided a better fit for the given dataset. Future work will explore additional hyperparameter tuning, alternative models, and feature engineering to further enhance predictive accuracy.

Summary:

By fitting and comparing both models, it was evident that SVM provided a better fit for the given dataset. Future work will explore additional hyperparameter tuning, alternative models, and feature engineering to further enhance predictive accuracy.

Future Work:

To improve the predictive accuracy of the models used in this analysis, several future steps could be taken:

Feature Engineering: Additional feature engineering could be explored to extract more meaningful information from the dataset. This could include creating polynomial features or applying non-linear transformations to existing variables.

Alternative Models: Investigating other model architectures, such as Random Forests, or Neural Networks, could provide better performance.

Cross-Validation Techniques: Employing more robust cross-validation techniques, such as k-fold cross-validation with a larger value of k, could provide more reliable estimates of model performance.

Feature Selection: Applying feature selection methods, such as recursive feature elimination or LASSO-based selection, could help reduce model complexity and improve interpretability without sacrificing performance.

R codes:

```
# Load Required Libraries

library(caret)

library(glmnet)

library(e1071)

library(data.table)

library(Matrix)

library(ggplot2)


# Load the Data

covid_19_data <- read.csv("covid_19_data.csv", stringsAsFactors = FALSE)


# Convert 'ObservationDate' to Date Type

covid_19_data$ObservationDate <- as.Date(covid_19_data$ObservationDate, format
= "%m/%d/%Y")


# Define the Split Ratio and Split the Data

split_ratio <- 0.8

split_index <- floor(nrow(covid_19_data) * split_ratio)
```

```
train_data <- covid_19_data[1:split_index, ]

test_data <- covid_19_data[(split_index + 1):nrow(covid_19_data), ]


# Check the Sizes of the Training and Testing Sets

cat("Training data size:", nrow(train_data), "Testing data size:", nrow(test_data), "\n")


# Identify Categorical Columns (Adjust Based on Your Data)

categorical_columns <- c('Province.State', 'Country.Region')


# Convert Categorical Variables to Dummy Variables

dummy_model <- dummyVars("~ .", data = train_data[, categorical_columns],
fullRank = TRUE)

train_data_dummy <- predict(dummy_model, newdata = train_data)

test_data_dummy <- predict(dummy_model, newdata = test_data)


# Combine Dummy Variables with the Rest of the Dataset (Excluding Categorical
Ones)

numeric_columns <- setdiff(names(train_data), categorical_columns)
```

```
train_data_numeric <- cbind(train_data_dummy, train_data[, numeric_columns])

test_data_numeric <- cbind(test_data_dummy, test_data[, numeric_columns])


# Remove Zero or Near-Zero Variance Predictors

nzv <- nearZeroVar(train_data_numeric)

train_data_numeric <- train_data_numeric[, -nzv]

test_data_numeric <- test_data_numeric[, -nzv]


# Impute Missing Values (Median Imputation)

preprocess_model <- preProcess(train_data_numeric, method = "medianImpute")

train_data_imputed <- predict(preprocess_model, newdata = train_data_numeric)

test_data_imputed <- predict(preprocess_model, newdata = test_data_numeric)


# Remove Highly Correlated Predictors

corr_matrix <- cor(train_data_imputed, use = "pairwise.complete.obs")

high_cor <- findCorrelation(corr_matrix, cutoff = 0.9)

train_data_reduced <- train_data_imputed[, -high_cor]

test_data_reduced <- test_data_imputed[, -high_cor]
```

```
# Scale the Data (Center and Scale)

preprocess_model <- preProcess(train_data_reduced, method = c("center", "scale"))

train_data_scaled <- predict(preprocess_model, newdata = train_data_reduced)

test_data_scaled <- predict(preprocess_model, newdata = test_data_reduced)


# Define Response Variable

y_train <- train_data_scaled$Confirmed

y_test <- test_data_scaled$Confirmed


# Remove Response Variable from Predictors

x_train <- train_data_scaled[, !names(train_data_scaled) %in% 'Confirmed']

x_test <- test_data_scaled[, !names(test_data_scaled) %in% 'Confirmed']


# Sample a Smaller Portion of the Dataset for Training

set.seed(123)

sample_size <- 5000  # Adjust as needed

sample_indices <- sample(seq_len(nrow(x_train)), size = sample_size)

x_train_sample <- x_train[sample_indices, ]

y_train_sample <- y_train[sample_indices]
```

```
# Remove Zero Variance Predictors

zero_var_cols <- nearZeroVar(x_train_sample)

x_train_sample <- x_train_sample[, -zero_var_cols]

x_test <- x_test[, colnames(x_train_sample)]  # Ensure Test Data Has the Same
Columns


# Apply PCA to Reduce Dimensionality (Retain 95% Variance)

preprocess_pca <- preProcess(x_train_sample, method = "pca", thresh = 0.95)

x_train_pca <- predict(preprocess_pca, x_train_sample)

x_test_pca <- predict(preprocess_pca, x_test)


# Define Cross-Validation Settings

train_control <- trainControl(method = "cv", number = 5)


# Train SVM Model on the PCA-Transformed Data

set.seed(123)

svm_model <- train(

  x = x_train_pca,
```

```
  y = y_train_sample,
  method = "svmRadial",
  trControl = train_control,
  preProcess = c("center", "scale")
)

# Make Predictions on the Test Set
svm_predictions <- predict(svm_model, newdata = x_test_pca)

# Evaluate Model Performance (For Regression)
rmse_value <- RMSE(svm_predictions, y_test)
mae_value <- MAE(svm_predictions, y_test)

cat("SVM Model RMSE:", rmse_value, "\n")
cat("SVM Model MAE:", mae_value, "\n")

# Train Other Models for Comparison

# Lasso Regression
```

```
set.seed(123)

lasso_model <- train(

  x = x_train_sample,

  y = y_train_sample,

  method = "glmnet",

  trControl = trainControl(method = "cv", number = 5),

  tuneGrid = expand.grid(alpha = 1, lambda = seq(0.001, 0.1, by = 0.005))

)


# Elastic Net Regression

set.seed(123)

elastic_net_model <- train(

  x = x_train_sample,

  y = y_train_sample,

  method = "glmnet",

  trControl = trainControl(method = "cv", number = 5),

  tuneGrid = expand.grid(alpha = seq(0, 1, length = 10), lambda = seq(0.001, 0.1, by =
0.005))

)
```

```
# Evaluate Models on Test Set

# Ensure Test Data Has the Same Columns as Training Data

x_test_reduced <- x_test[, colnames(x_train_sample)]


# Lasso Predictions

lasso_preds <- predict(lasso_model, x_test_reduced)

lasso_rmse <- RMSE(lasso_preds, y_test)


# Elastic Net Predictions

elastic_net_preds <- predict(elastic_net_model, x_test_reduced)

elastic_net_rmse <- RMSE(elastic_net_preds, y_test)


# Print RMSE Values for Comparison

cat("Lasso RMSE:", lasso_rmse, "\n")

cat("Elastic Net RMSE:", elastic_net_rmse, "\n")

cat("SVM RMSE:", rmse_value, "\n")


# Summarize Model Performances
```

```
results <- resamples(list(Lasso = lasso_model, ElasticNet = elastic_net_model, SVM =
svm_model))

summary(results)
```